NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

GROUP ASSIGNMENT REPORT

Midterm

Web Design and Programming (Course code)

Revenue Analytics Dashboard

A web-based mini BI tool for CSV revenue data: upload, visualize and forecast

Group: Group 06
Class: AI66A
Instructor: M.Sc. Le Duy Khanh
Semester: Semester 1, 2026–2027
GitHub: https://github.com/AI-NATV/Group6_Web_Design

Team Members

No.	Full name	Student ID	Role
1	Duong Dinh Anh	11247256	Team lead, Back-end
2	Tran Khai Van	11247368	Front-end, UI/UX
3	Nguyen Thi Nhien	11247337	Database, testing
4	Nguyen Hoang Tuan	11247363	Documentation, deployment

Hanoi, November 2026

Declaration

We declare that this report is the result of our group's own work for the course *course*. All sources (literature, source code, images, AI tools) are cited explicitly. We take full responsibility for the submitted content.

Task Allocation and Contributions

Member	Work done	Contribution (%)
Duong Dinh Anh	API design, authentication, repository management	25%
Tran Khai Van	Interface design, responsive layout, CSS/JS	25%
Nguyen Thi Nhien	Database design, tests, integration testing	25%
Nguyen Hoang Tuan	Report writing, deployment, demo video	25%
	Total	**100%**

Table of Contents

1 Introduction . . . 5
1.1 Background and motivation . . . 5
1.2 Objectives . . . 5
1.3 Scope and target users . . . 5
1.4 Technologies . . . 5
1.5 Report structure . . . 5
2 Analysis and Design . . . 6
2.1 Requirements . . . 6
2.1.1 Functional requirements . . . 6
2.1.2 Non-functional requirements . . . 6
2.2 Interface design (UI/UX) . . . 7
2.3 System architecture . . . 7
2.4 Database design . . . 8
2.5 API design . . . 8
3 Implementation . . . 8
3.1 Project structure . . . 8
3.2 User interface . . . 9
3.3 Server-side logic . . . 9
3.4 Data processing . . . 10
3.5 Baseline forecasting . . . 10
3.6 Feature assistant . . . 10
3.7 Source control and teamwork . . . 11
3.8 Deployment . . . 11
4 Results and Conclusion . . . 11
4.1 Testing . . . 11
4.2 Achievements and limitations . . . 12
4.3 Lessons learned and future work . . . 13
4.4 Conclusion . . . 13
References . . . 14
A Appendix . . . 14
A.1 Setup and run instructions . . . 14
A.2 Tests . . . 14
A.3 Demo accounts . . . 14

A.4 Use of AI tools 14

List of Figures

1 Client–server architecture of the application. 7

List of Tables

1 Technologies used in the project. 6
2 Database tables. 8
3 Main REST endpoints. Dataset and visual routes require a Bearer token. 8
4 Selected test cases. 12

1. Introduction

1.1. Background and motivation

Small businesses and student teams often keep their sales records as CSV exports from spreadsheets, point-of-sale systems or online shops. Such files are easy to produce but slow to analyse. A spreadsheet can draw a chart, but it does not keep a saved dashboard for each dataset, it does not separate the data of different users, and it offers no systematic way to test whether a forecast works on data it has not seen.

Revenue Analytics Dashboard is a small web application that addresses these gaps. A user uploads a CSV file, reviews how the application has interpreted each column, builds charts that are stored on a dashboard, and evaluates forecasts with chronological validation. The project brings together the main parts of a web application: a multi-page user interface, a documented REST API, authentication and authorization, a relational database, and automated tests.

1.2. Objectives

1. Accept CSV files up to 10 MB, recognise the type and role of each column automatically, and let the user correct the result. Every upload is either saved with a review or rejected with a clear message.
2. Let users create bar, line, pie and scatter charts from their own columns, using five aggregations (sum, average, count, minimum, maximum), and save them to a dashboard that keeps a user-defined order.
3. Provide forecasts that are compared on two validation windows and a held-out test set, and report MAE and RMSE against a naive baseline. Optional historical numeric features are added only when the user confirms them and they win on validation.
4. Keep each user's data private: every dataset and visual route requires a valid token and checks ownership.

1.3. Scope and target users

The target users are analysts, students and small-business owners who hold one or more CSV files of dated or categorical records, plus administrators who manage accounts.

In scope: account registration and login; CSV upload with validation; normalization of currency, percentage and date values; semantic review with user overrides; data-quality counts and cleaning; drag-and-drop chart building; saved dashboards; filtering one product or group; forecasting with backtesting; explainable feature suggestions based on training history only; English and Vietnamese interface.

Out of scope: future or categorical predictors; multivariate models such as ARIMAX or VAR; causal analysis; database engines other than SQLite and PostgreSQL; password reset and e-mail verification; a user interface for managing roles; connectors to live databases or accounting software; native mobile applications.

1.4. Technologies

1.5. Report structure

Section 2 covers the requirements and design decisions. Section 3 describes the implementation, including the most important code paths and the feature assistant. Section 4 presents the testing approach, the results, the limitations and future work. The appendix contains setup instructions.

Component	Technology	Purpose
Front-end	HTML5, CSS3, vanilla JavaScript	User interface, no build step
Charts	Chart.js 4.5.1 (CDN)	Bar, line, pie and scatter charts
Back-end	Python 3.11+, FastAPI 0.162.0	REST API, validation, OpenAPI docs
Data	SQLModel 0.0.48, pandas 3.0.5	Table models, CSV parsing and profiling
Learning	scikit-learn 1.8.0	Ridge regression, imputation and scaling for optional features
Security	pwdlib with bcrypt, PyJWT 2.15.0	Password hashing, signed login tokens
Database	SQLite (default), PostgreSQL	Storage of users, datasets and visuals
Migrations	Alembic 1.13 or later	Versioned database schema
Testing	pytest 9.2.0, httpx	Automated API tests
Tooling	Git, GitHub	Version control and collaboration

Table 1. Technologies used in the project.

2. Analysis and Design

2.1. Requirements

2.1.1. Functional requirements

FR1. Users can register an account, sign in and sign out.

FR2. Users can upload a CSV file. The system rejects files that are not CSV, empty, larger than 10 MB, header-only, or have blank or duplicate column names.

FR3. Users can preview the uploaded data page by page, showing both the original text and the normalized values.

FR4. The system recommends a type and a role for each column (for example date, identifier, category or measure). Users can override every recommendation, and the override is saved on the server.

FR5. The system reports data-quality counts and lets the user remove exact duplicate rows or rows with missing values.

FR6. Users build a chart by dragging fields into compatible wells for bar, line, pie or scatter charts.

FR7. Users save charts to the dashboard of a dataset, reorder them, remove them, and see KPI cards.

FR8. Users choose a time column and a numeric target, a horizon of 1 to 90 periods, and a frequency; the system then compares forecasting methods and shows future values.

FR9. When a file contains several products or groups, users choose one product column and one series, so only the selected rows are used for training.

FR10. The system suggests numeric features using only the initial training history. Users must confirm the features they accept, and the extra model is used only if it wins on validation.

FR11. Each user sees only their own datasets. Administrators can see all datasets.

2.1.2. Non-functional requirements

- **Security:** passwords are stored only as salted bcrypt hashes [7]; login tokens expire after 30 minutes by default; the owner of an upload and the role of a user are decided by the server, never by the request; all rendered data is escaped.
- **Performance:** the dataset list does not transfer row data; uploads are read only up to the size limit plus one byte; forecasts and feature review are limited to 50,000 rows, and feature review to 100 columns.

- **Consistency:** the database is the source of truth. The browser keeps only a copy of the active dataset for navigation between pages, and every change is saved to the API first.
- **Usability:** the three workspaces share one navigation bar and show the active dataset in the header. The interface is available in English and Vietnamese.
- **Responsiveness:** the layout must work on a phone-sized screen. This is covered by a browser check in the project scripts.
- **Maintainability:** request and response schemas are typed, and the API is documented automatically with OpenAPI.

2.2. Interface design (UI/UX)

The application has a sign-in page, a separate registration page, and three workspaces:

- **Data:** upload a file or choose a saved dataset, review the overview, the preview, the semantic review and the data quality.
- **Visualize:** choose a chart type, drag fields into the wells, preview the chart, and manage the saved dashboard.
- **Forecast:** configure the time column, target, horizon and frequency, optionally filter one product or group, review feature suggestions, run the evaluation, and read the future forecast table. Results can be downloaded as a JSON report.

Users can move between workspaces without losing the active dataset, because it is kept in the browser session.

Screenshots: [to be inserted by the group: desktop and mobile views of each workspace].

2.3. System architecture

The system follows a client–server model. The browser loads static HTML, CSS and JavaScript. The same FastAPI [8] application serves these files under `/app` and exposes the REST API under `/api/v1`. Figure 1 shows the main components.

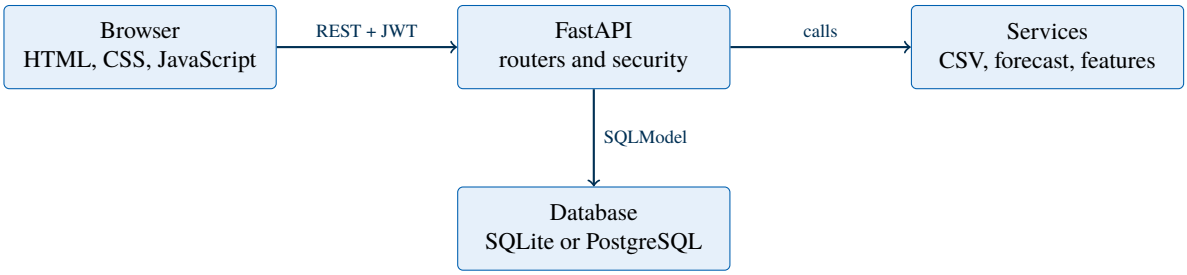


Figure 1. Client–server architecture of the application.

Three design decisions shape the flow. First, datasets are saved on the server, so a user can return from a new browser and find them again. Second, the Forecast page works on the saved, normalized dataset through an authenticated endpoint, so the server checks ownership and the browser cache is not trusted for analysis. Third, the original stateless endpoint `POST /forecast` remains for baseline comparison: the client sends the observations, the server computes the result, and nothing is stored.

2.4. Database design

The database has three tables. One user owns many datasets, and one dataset has many visuals. Deleting a user's dataset also deletes its visuals.

Table	Main columns
`user`	`id`, `username` (unique, lower case), `hashed_password`, `role` (`user` or `admin`), `created_at`
`dataset`	`id`, `owner_id` (foreign key to user), `filename`, `rows`, `columns` (JSON), `data` and `raw_data` (JSON), `profiles` and `semantic_model` (JSON), `created_at`, `updated_at`
`visual`	`id`, `dataset_id` (foreign key to dataset), `chart_type`, `fields` (JSON), `aggregation`, `position`, `created_at`

Table 2. Database tables.

The tables are created by the initial Alembic migration (revision `20261103_01`), so the schema is under version control. The feature assistant does not add tables: its suggestions and results are computed on demand and are not stored.

Two choices are worth noting. The `raw_data` column keeps the original text of each cell, so the Data workspace can show exactly what was uploaded, while `data` keeps the normalized values used for analysis. The `profiles` and `semantic_model` columns are JSON because each column has a different set of properties.

2.5. API design

Method	Endpoint	Purpose
POST	`/api/v1/register`	Create an account (201)
POST	`/api/v1/token`	Sign in with form fields; returns a Bearer token
GET	`/api/v1/me`	Current user
POST	`/api/v1/datasets`	Upload a CSV (201)
GET	`/api/v1/datasets?offset=&limit=`	List own datasets, newest first
GET	`/api/v1/datasets/{id}`	Read one dataset
PATCH	`/api/v1/datasets/{id}`	Update semantic review or cleaned rows
DELETE	`/api/v1/datasets/{id}`	Delete a dataset and its visuals (204)
GET	`/api/v1/datasets/{id}/visuals`	List visuals in dashboard order
POST	`/api/v1/datasets/{id}/visuals`	Save a visual at the end (201)
PATCH	`/api/v1/visuals/{id}`	Move a visual to a new position
DELETE	`/api/v1/visuals/{id}`	Remove a visual (204)
POST	`/api/v1/datasets/{id}/feature-review`	Suggest features from initial training history only
POST	`/api/v1/datasets/{id}/forecast`	Compare baselines and confirmed features on a saved dataset
POST	`/forecast`	Stateless baseline forecast (200)

Table 3. Main REST endpoints. Dataset and visual routes require a Bearer token.

Errors follow the same pattern across the API: 401 means the user is not signed in or the token is invalid, 403 means the dataset belongs to another user, 404 means the resource does not exist, 400 or 422 means the request is invalid, and 413 means the file is too large. The 401 and 403 cases are separated on purpose, so the client knows whether to ask the user to sign in again.

3. Implementation

3.1. Project structure

```
Group6_Web_Design/
|-- .env.example            # configuration template (no secrets)
|-- alembic.ini, migrations/  # Alembic configuration and schema migrations
|-- sample_sales.csv        # 48-row sample data for the Data workspace
|-- backend/
|   |-- main.py             # FastAPI app, middleware, /health, /forecast
|   |-- config.py           # environment settings
|   |-- database.py         # engine, table creation, session dependency
|   |-- db_models.py        # tables and API schemas
|   |-- security.py         # bcrypt, JWT, access checks
|   |-- models.py           # forecast request and response contracts
|   |-- routers/            # auth, datasets, visuals, forecast
|   |-- services/           # csv_service, forecast_service, feature_service
|   |-- seed.py, create_admin.py
|   `-- tests/              # pytest suite (six files)
|-- frontend/
|   |-- login.html, register.html, index.html, visualize.html, forecast.html
|   |-- css/style.css
|   `-- js/                 # api, auth, state, semantic, ui, i18n and pages
|-- scripts/check_forecast_browser.cjs
|-- requirements.txt
`-- README.md
```

3.2. User interface

Each page loads the same shared scripts: `api.js` for requests, `auth.js` for the session, `state.js` for the cached dataset, `semantic.js` for column recognition and `ui.js` for common helpers. Each workspace then has its own script. The Visualize and Forecast pages load Chart.js [1] from a CDN, and the pages are written in HTML, CSS and JavaScript [6, 2].

Screenshots: [to be inserted by the group].

3.3. Server-side logic

Every protected route depends on `get_current_user`, which reads the Bearer token, verifies its signature and expiry [4], and reloads the user from the database (Listing 1). Reloading the user means that a deleted account is locked out at once and that a role cannot be changed by editing a token. The API itself is built with FastAPI [8].

Listing 1. Token check in backend/security.py (simplified).

```python
def get_current_user(token: Annotated[str, Depends(oauth2_scheme)],
                     session: SessionDep) -> User:
    try:
        payload = jwt.decode(token, require_secret_key(),
                             algorithms=[ALGORITHM],
                             options={"require": ["exp", "sub"]})
        user_id = int(payload["sub"])
    except (jwt.InvalidTokenError, KeyError, ValueError):
        raise CREDENTIALS_ERROR
    user = session.get(User, user_id)
    if user is None:
        raise CREDENTIALS_ERROR
    return user
```

The baseline forecast service selects the model with the lowest validation error and uses the held-out test

only for reporting, as Listing 2 shows.

Listing 2. Model selection in backend/services/forecast$_service.py$.

```
selected = min(comparison, key=lambda item: item["validation"]["mae"])["model"]
test_predictions, _ = predict(selected, values[:test_start], horizon,
                              request.season_length)
```

Because the choice uses only validation folds, changing the test values cannot change which model is selected. A dedicated test checks this property.

3.4. Data processing

When a CSV file is uploaded, `parse_csv` (built on pandas [5]) processes each column in three steps:

1. Missing markers such as an empty cell, `-`, `n/a` or `null` are treated as missing values.
2. The column is tested as a number. The parser accepts currency symbols, percentages, accounting negatives in brackets, and both comma and point as decimal separators. When both separators appear, the right-most one is treated as the decimal separator, so a price such as $123.456 is not corrupted.
3. The column is tested as a date in several formats. If a slash date such as `03/04/2026` can be read as month-first or day-first, the parser keeps the raw text instead of guessing.

Columns whose names suggest identifiers (for example `customer_id` or `zip`) are never treated as measures. The original text is always kept in `raw_data`.

3.5. Baseline forecasting

The baseline forecast uses four methods: last-value Naive, the mean of the last seven observations, damped Holt trend, and optional Seasonal Naive. Holt's smoothing parameters are chosen from a small fixed grid using training residuals only. This is a simplified method, not a full ETS optimizer, which is consistent with the textbook treatment of exponential smoothing [3].

For a horizon h, the last $3h$ observations form two consecutive validation windows and one untouched test window. Each validation forecast uses an expanding training prefix and predicts all h steps without seeing the intervening actual values. The model with the lowest pooled validation MAE is selected, and ties favour Naive. The selected model is then scored on the test window and refitted on all history to produce the future forecast. The eligibility rule requires at least $\max(40, \max(20, 2s) + 3h)$ usable dates, where s is the season length; this is a product guardrail, not a statistical guarantee.

3.6. Feature assistant

The newest addition is an optional, explainable feature assistant. It is a rules-and-statistics tool, not a language model, and it uses no external AI service. It works in two steps on a saved dataset that the user owns.

1. **Feature review.** The review looks only at the initial training prefix, which is the data before both validation windows and the test window. Each numeric column is classified. Identifiers and sensitive names (for example `id`, `email`, `phone`), names suggesting future outcomes (for example `forecast`, `predicted`), target copies, constants, non-numeric columns and columns with more than 20% missing values are excluded. The strongest lag-1 correlations, with absolute value at least 0.2, are suggested, up to three columns.
2. **Confirmed forecast.** The user selects at most eight numeric features and confirms them. Ridge regression

with fixed regularization ($\alpha = 1$) [9] is then compared with the baselines on the same two validation windows. Each training sample uses a snapshot taken at the forecast origin: lagged target values at 1, 2, 3 and 7 steps, a trailing seven-step mean, and the last observed value of each selected feature. The model predicts the entire horizon from that snapshot, so it never uses feature or target values from inside the horizon it is predicting. Imputation and scaling are fitted anew on each training prefix, never on validation or test data.

Only the model selected on validation is scored against Naive on the untouched test window. If a baseline wins, the final forecast does not use the extra features. The results also report descriptive insights: historical minimum and maximum, the largest consecutive change, and the final predicted change compared with the last actual value. These statistics describe the data; they do not imply causation.

A review token ties the confirmation to the current dataset revision and settings. Changing the settings or updating the saved dataset invalidates the review, so a stale confirmation cannot be used.

3.7. Source control and teamwork

The team worked in feature branches and merged each one through a pull request on GitHub (`https://github.com/AI-NATV/Group6_Web_Design`). Three pull requests were merged into `main`: #1 (frontend checklist fixes for weeks 2–5), #2 (storing datasets and dashboards in a database) and #3 (user accounts, JWT login and per-user datasets). Later work, including the migration, the registration page and the feature assistant, was committed directly to `main`. The history contains twelve commits from two GitHub accounts. [Map each GitHub account to a member name in the contribution table.]

3.8. Deployment

The application runs locally with the commands given in the appendix (Section A.1). The feature assistant requires scikit-learn, which is installed by `requirements.txt`. For a shared deployment the group should set `ALLOWED_ORIGINS` to the HTTPS frontend origin, configure a PostgreSQL `DATABASE_URL`, and keep `SECRET_KEY` in the environment. The public demo address is listed on the cover page.

4. Results and Conclusion

4.1. Testing

The backend has an automated pytest suite of six files: API behaviour, authentication, dataset and visual handling, forecasting, permissions, and the feature assistant. Each test runs against a fresh in-memory database, so tests never touch the development data. The feature assistant tests check that suggestions use only the initial training window, that confirmation is required, that a dataset change invalidates a review, and that the assisted routes enforce login and ownership. A browser check script additionally uploads a synthetic 120-row CSV through the interface and checks the forecast tables and charts, the date order, and the layout at mobile width. That script is run manually.

On the latest `main` branch (commit `b94f2d4`), running `python -m pytest backend/tests -q` gives **109 passed** tests, with one deprecation warning from the test client library. The warning does not affect the results.

Table 4 lists the selected test cases.

Performance measurements (for example Lighthouse scores for Performance, Accessibility, Best Practices and SEO) have not yet been recorded. [Add the scores and a screenshot once measured.]

ID	Feature (test)	Expected result	Result
TC01	Registration stores only a salted hash	201; no plain password stored	Pass
TC02	Wrong password and unknown user	Same 401 response for both	Pass
TC03	Expired, tampered or foreign token	401 Unauthorized	Pass
TC04	Role is read from the database	Token cannot grant admin rights	Pass
TC05	Non-CSV or header-only upload	400 with a clear message	Pass
TC06	Currency normalization on upload	Numbers parsed; raw text kept	Pass
TC07	Other user accesses a dataset	403 on every dataset action	Pass
TC08	Test values change model selection	Selection and test predictions unchanged	Pass
TC09	Move and delete visuals	Positions stay contiguous from 0	Pass
TC10	Feature review ignores test and future data	Same suggestions with test values changed	Pass
TC11	Assisted routes need login and ownership	401 or 403 for other users and anonymous requests	Pass

Table 4. Selected test cases.

4.2. Achievements and limitations

Compared with the objectives in Section 1:

- **Objective 1 (upload and review):** implemented. Uploads are validated, columns are recognised, and user overrides are saved on the server.
- **Objective 2 (charts and dashboard):** implemented for bar, line, pie and scatter charts with five aggregations and ordered dashboards.
- **Objective 3 (forecasting):** implemented with two validation windows, a held-out test, and MAE and RMSE against the Naive baseline. The Holt method is simplified. Optional historical features are used only after user confirmation and only if they win on validation.
- **Objective 4 (privacy):** implemented through tokens, the database check on every request, and ownership checks on every dataset and visual route, including the new forecast routes.

Known limitations:

- Login tokens are stored in `sessionStorage`. Any script injected into a page could read them, so the application depends on escaping all rendered data.
- Access tokens cannot be revoked before they expire, because there are no refresh tokens or server-side logout. The lifetime is therefore kept short (30 minutes by default).
- There is no password reset and no rate limiting on login attempts.
- The initial Alembic migration creates the current three-table schema. Each later change to the models needs a new, reviewed migration before it is deployed.
- Import uses heuristics for comma-only numeric notation and for ambiguous dates, so users must review the data before forecasting.
- Charts and the Chart.js library require a network connection to the CDN.
- Forecasting handles one date/target series per run, optionally filtered to one product or group, and gives point estimates only. ARIMA, future or categorical predictors, simultaneous multi-series training, prediction intervals and saved models are not implemented.
- Historical features cannot be checked against when each value was actually published. Name heuristics cannot detect every leakage source, so the suggestions still need human review.
- Seasonality must be given by the user; it is not inferred.

4.3. Lessons learned and future work

Several lessons came from the development process:

- Numeric parsing needs explicit rules. A rule that removed every decimal point made prices such as $123.456 one thousand times too large, which was corrected by treating the right-most separator as decimal.
- bcrypt ignores everything after 72 bytes, and accented Vietnamese letters take up to three bytes each. Validation therefore counts bytes, not characters.
- Keeping the server as the single source of truth simplified the three pages, at the cost of one API call per change.
- In forecasting, every preprocessing step that learns from data (imputation, scaling, model selection) must be fitted only on the training prefix. Otherwise test data leaks into the result, and the test score stops being an honest estimate.

Future work could include: automatic migration checks in continuous integration; HTTP-only cookies for tokens; automatic season detection; more forecasting methods with full Holt-Winters fitting; exports of the dashboard to PDF; saving forecast runs; and measured accessibility and performance results.

4.4. Conclusion

The project delivers a working mini BI application that covers the full path from a raw CSV file to a saved dashboard and a validated forecast. The architecture separates the user interface, the REST API, the business services and the database. The automated tests, now 109 passing, protect the most important rules for accuracy and privacy, and the feature assistant adds an explainable, leakage-aware option for forecasting with historical features.

References

[1] Chart.js Contributors. Chart.js documentation, 2026. Accessed: 2026-11-02.

[2] Jon Duckett. *HTML and CSS: Design and Build Websites*. Wiley, 2011.

[3] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 3rd edition, 2021.

[4] M. Jones, J. Bradley, and N. Sakimura. Json web token (jwt). RFC 7519, IETF, 2015.

[5] Wes McKinney. Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference*, 2010.

[6] Mozilla. Mdn web docs, 2026. Accessed: 2026-11-02.

[7] Niels Provos and David Mazières. A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference*, 1999.

[8] Sebastián Ramírez. Fastapi documentation, 2026. Accessed: 2026-11-02.

[9] scikit-learn Developers. scikit-learn: Machine learning in python, 2026. Accessed: 2026-11-05.

A. Appendix

A.1. Setup and run instructions

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python -c "import secrets; print(secrets.token_hex(32))"
$env:SECRET_KEY="paste-the-generated-value"
alembic upgrade head
uvicorn backend.main:app --reload
```

Then open `http://127.0.0.1:8000/app/` in a browser. The API documentation is available at `http://127.0.0.1:8000/docs`.

A.2. Tests

```
python -m pytest backend/tests -q
```

A.3. Demo accounts

Users create their own accounts on the registration page (`register.html`, linked from the sign-in page). An administrator account is created from the command line with `python -m backend.create_admin USERNAME`. No shared password is listed in this report.

A.4. Use of AI tools

[List any AI tools the group used, for what purpose, and which parts of the product or report they helped with.]

REVENUE ANALYTICS DASHBOARD

Implementation and Verification Update

Group 06 | Web Design and Programming
Status date: 10 November 2026

Purpose of this addendum. This update is appended to the report `Group6_Report_LaTeX__1_.pdf`. The original PDF is retained unchanged because its editable LaTeX source was not present in the project. This addendum records the implemented state and supersedes earlier report statements wherever they conflict with the current code, especially the database schema, chart interaction, feedback, account functions, migration status, and test count.

1. Current product scope

Revenue Analytics is a browser-based mini BI application for students, small sellers or teams, and analysts who have product or sales data in CSV files. It supports dataset review and preparation, interactive charts, saved dashboards, and forecasts evaluated against held-out historical data. It is not a live market feed or an automated decision system.

- **Data workspace:** validates CSV uploads (maximum 10 MiB), previews original and normalized cell values, recommends column types and roles, and reports missing values and exact duplicate rows. Common date, number, currency, and percentage formats are normalized where unambiguous. Removing duplicate or incomplete rows is an explicit user action; the importer does not silently delete them.
- **Visualize workspace:** supports Bar, Line, Pie, and Scatter charts, SUM, AVERAGE, COUNT, MIN, and MAX aggregation, dashboards with saved/reordered/ removable visuals, and touch-friendly field dropdowns as an alternative to drag-and-drop.
- **Forecast workspace:** evaluates one selected numeric time series, optionally filtered to a product or group. It shows model comparison, held-out performance, future point estimates, data warnings, and saved runs.

2. User interface and interaction updates

The sign-in page is also the bilingual project landing page. It now introduces the project, its intended users, supported forecasting methods, and the purpose and limitations of the application. The hero illustration is positioned below the bold heading without rotation and scales to narrow screens. The landing page includes a public feedback form.

On phones and tablets, chart fields can be assigned from dropdowns within each field well; dragging is optional. When the dataset has usable ISO date values, all chart types provide a searchable date checklist with *Select all*, *Select none*, and individual date selection. A selection immediately rebuilds the active chart and is retained when the chart type changes. Saved visuals persist the selected date column and dates, so the dashboard recreates the same filtered view.

A Pie chart has one numeric measure and divides its aggregate into category slices. This is appropriate for showing composition; Bar or Line charts are available when comparing measures or trends. The interface explains this distinction rather than suggesting that a pie chart should accept multiple numeric measures.

The field-search input disables browser autocomplete and is cleared when the Visualize page initializes. This prevents a stored email or other browser suggestion from filtering the field list and making the dataset appear absent. The selected CSV filename is shown before upload. Responsive layout was checked in browser viewports from 320 px to 1440 px; this is viewport emulation, not a claim of testing on every physical handset. Updated asset versions and revalidation of served HTML reduce the chance of showing a cached older UI on the first page load. Incorrect-password feedback for account deletion appears beside the password field and clears when the user retries.

3. Accounts, privacy, and feedback

There is no separate username: registration and sign-in use email. For the OAuth2 password-form endpoint, the email is submitted in the field named `username`. The backend trims and lowercases email addresses during registration and sign-in, so email capitalization does not change account identity. Passwords remain case-sensitive and are stored as bcrypt hashes. Login tokens are JWTs signed using the required `SECRET_KEY`.

Signed-in users can edit an optional display name and phone number, upload a PNG/JPEG/WebP avatar up to 140 KiB, choose a per-browser light/dark/system theme, and delete an account after password verification and confirmation. Dataset ownership and roles are determined by the server.

Feedback is stored independently of accounts. Administrators can review it through the authenticated `GET /api/v1/admin/feedback` endpoint; the web interface does not currently include an inbox. A regular user cannot use the administrator-only endpoint.

4. Data storage and current database schema

The default database is SQLite at `data/app.db`; deployments can select another database such as PostgreSQL through `DATABASE_URL`. The server stores parsed dataset rows, not a byte-for-byte copy of the uploaded CSV file. Each dataset keeps original cell text, normalized analysis values, profiling metadata, and the user's final semantic review. The browser keeps an active working copy and token in `sessionStorage`; the database remains the durable source of truth.

Table	Stored information
`user`	Email, password hash, role, optional profile fields, avatar, and creation time.
`dataset`	Owner, filename, row and column information, original and normalized values, profiles, and semantic review.
`visual`	Dataset, chart type, field mapping, aggregation, optional `date_filter` JSON, display order, and creation time. It does not store a chart image.
`forecastrun`	Owner and dataset, forecast settings, selected model/features, and saved result.
`feedback`	Submitter name, email, message, and creation time; readable only by administrators.

A visual is reconstructed from its dataset and saved chart settings. Deleting a dataset cascades to its visuals and forecast runs. The optional date filter records a date-column name and a unique list of selected `YYYY-MM-DD` values; older visuals without a filter continue to display all dates.

5. Forecasting and interpretation

Available methods are last-value Naive, the mean of up to the last seven observations, damped Holt trend, optional Seasonal Naive when a season length is specified, and optional Ridge regression when the user reviews and confirms eligible historical numeric features. Ridge uses lagged target values and selected historical feature values; it does not use actual values from inside the forecast horizon. Imputation and scaling are fitted on training samples.

The forecast horizon is 1–90 periods. Two expanding chronological validation windows select the method by pooled MAE. The final held-out test window is kept separate from model selection; MAE and RMSE are reported, then the selected method is refitted on usable history to produce future point estimates. Each assisted run is saved for later review. The system does not currently provide prediction intervals, guarantee future results, infer seasonality, or support simultaneous multi-series forecasting.

6. Migrations, setup, and verification

The current schema includes incremental Alembic revisions `20261109_05` (feedback storage) and `20261110_06` (optional saved-visual date filter). Revision `20261110_06` depends on `20261109_05`; therefore, the complete `migrations/versions/` directory must accompany the source code. For a new or existing database, run `python -m alembic upgrade head` from the project root before starting the application. Existing databases should be backed up before schema migration. The application requires `SECRET_KEY` at startup.

On 10 November 2026, the backend suite completed with **135 passed**. JavaScript syntax checks, `git diff -check`, browser checks for field search and date filtering, and an isolated SQLite migration/startup check also passed. The isolated checks used temporary databases; they did not replace project data.

7. Limitations and follow-up

CSV normalization uses heuristics, so ambiguous dates and locale-dependent numbers require review. Missing periods are not automatically imputed for forecasting. Forecasts are point estimates from historical patterns, not business guarantees. Feedback review is API-only, and chart rendering depends on the Chart.js CDN unless the library is bundled locally. Physical-device accessibility, production deployment, and real-user forecast quality still require separate evaluation.